

SESSION ELEVEN: PYTHON ESSENTIAL FOR DATA SCIENCE

D. K. MURIITHI

Objective

This session aims to deepen participants' foundational understanding of Python by introducing and reinforcing essential programming concepts and tools relevant for data science. Participants will move beyond the basics of variables and loops to explore how Python's built-in capabilities can be leveraged for effective data handling and preprocessing. The focus is on five key areas critical for any data science or analytics workflow:

1. **Advanced Data Types:** Understanding specialized Python containers such as namedtuple, defaultdict, and iterators from the itertools module allows for cleaner and more structured data management. These data types can enhance readability, performance, and reduce code complexity in real-world data processing tasks.
2. **Functional Programming:** The ability to use lambda, map, filter, and reduce enables participants to write concise, elegant code that is efficient for data transformations. These tools are particularly valuable when performing repetitive or conditional tasks on data elements.
3. **Comprehensions:** Mastering list, dictionary, and set comprehensions equips learners to transform and filter datasets compactly, a fundamental aspect of data wrangling and feature engineering.
4. **File Input/Output (I/O):** Interacting with external files such as text, CSV, and JSON formats is a daily necessity in data science. This session covers best practices for reading, writing, and parsing data using Python's standard libraries.
5. **Jupyter Notebooks:** As the de facto environment for data science work, understanding how to efficiently use Jupyter Notebooks ensures that participants can document, debug, and present their analyses interactively.

By the end of this session, participants will be equipped with the tools and techniques necessary to clean, transform, and manage data using core Python. These skills provide a strong foundation for more advanced data analysis and machine learning tasks in upcoming sessions.

Advanced Data Types

1. Collections

Python's `collections` module provides specialized container datatypes beyond built-in types like list, tuple, dict, and set.

- `Counter` : For counting hashable objects
- `defaultdict` : Like a dictionary, but provides a default value for the key that does not exist
- `OrderedDict` : A dict that remembers insertion order (now built-in in modern Python)
- `namedtuple` : Factory function for creating tuple subclasses with named fields
- `deque` : A list-like container with fast appends and pops on either end

The code below uses `Counter` to count how many times each fruit appears in the list. It then prints a dictionary-like output showing the frequency of each fruit.

```
In [1]: from collections import Counter, defaultdict, namedtuple, deque

# Counter example
cnt = Counter(['apple', 'orange', 'apple', 'pear', 'orange', 'banana'])
print(cnt)
```

```
Counter({'apple': 2, 'orange': 2, 'pear': 1, 'banana': 1})
```

This code uses a `defaultdict` with integers to automatically initialize missing keys to zero and count how many times each item appears. It prints a dictionary showing the frequency of each fruit without needing to check if a key already exists.

```
In [2]: # defaultdict example
d = defaultdict(int)
for item in ['apple', 'orange', 'apple']:
    d[item] += 1
print(d)
```

```
defaultdict(<class 'int'>, {'apple': 2, 'orange': 1})
```

This code creates a `namedtuple` called `Point` with fields `x` and `y`, allowing access to values by name instead of index. It then creates a point with coordinates `(1, 2)` and prints the `x` and `y` values.

```
In [3]: # namedtuple example
Point = namedtuple('Point', ['x', 'y'])
p = Point(1, 2)
print(p.x, p.y)
```

```
1 2
```

This code creates a `deque` with initial elements and adds an item to both the left and right ends. It then prints the updated deque, showing efficient insertion at both ends.

```
In [4]: # deque example
q = deque([1, 2, 3])
q.appendleft(0)
q.append(4)
print(q)
```

```
deque([0, 1, 2, 3, 4])
```

Functional Programming

Python supports functional programming paradigms using built-in functions like `map`, `filter`, `reduce`, and anonymous functions (`lambda`).

- **Map:** This code uses `map()` to apply a function that squares each number in the list, producing a new list of squared values.
- **Filter:** This code uses `filter()` to select only the even numbers from the list, returning a list that meets the condition.
- **Reduce:** This code uses `reduce()` to multiply all the numbers in the list together, resulting in a single value that is the product of all elements.

```
In [5]: from functools import reduce

# map example
nums = [1, 2, 3, 4, 5, 6, 7, 8, 10]
squares = list(map(lambda x: x**2, nums))
print("Squares:", squares)

# filter example
even = list(filter(lambda x: x % 2 == 0, nums))
print("Even Numbers:", even)

# reduce example
product = reduce(lambda x, y: x * y, nums)
print("Product:", product)
```

```
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 100]
```

```
Even Numbers: [2, 4, 6, 8, 10]
```

```
Product: 403200
```

Comprehensions

Comprehensions provide a concise way to create lists, dictionaries, and sets from iterables.

- **List comprehension:** This code creates a list containing the squares of numbers from 0 to 9 in a concise and readable way.
- **Dictionary comprehension:** This code creates a dictionary where each key is a number and its value is the square of that number.

- **Set comprehension:** This code creates a set of squared values, automatically removing any duplicate results.

```
In [6]: # List comprehension
squares = [x**2 for x in range(10)]
print("Squares:",squares)

# Dictionary comprehension
square_dict = {x: x**2 for x in range(5)}
print("Square Dictionary:", square_dict)

# Set comprehension
unique_squares = {x**2 for x in [1, 2, 3]}
print("Unique squares:",unique_squares)
```

```
Squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
Square Dictionary: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
Unique squares: {1, 4, 9}
```

File I/O

Python makes it easy to read and write files such as text, CSV, and JSON.

- **Text file I/O:** This code writes a short message to a text file and then reads the file contents back and prints them.

```
In [7]: # Writing and reading a text file
with open("example.txt", "w") as f:
    f.write("Hello, world!\nPython file I/O.")

with open("example.txt", "r") as f:
    print(f.read())
```

```
Hello, world!
Python file I/O.
```

- **JSON handling:** This code saves a Python dictionary to a JSON file and then loads it back into Python, preserving the data structure.

```
In [8]: # JSON
import json
data = {'name': 'Alice', 'age': 25, 'city': 'Nairobi'}
with open('data.json', 'w') as f:
    json.dump(data, f)

with open('data.json', 'r') as f:
    loaded = json.load(f)
    print(loaded)
```

```
{'name': 'Alice', 'age': 25, 'city': 'Nairobi'}
```

```
In [9]: import json
        json.dump({"name": "Dennis", "course": "Python for Data Science"}, open("data.json",
        "w"))

        with open('data.json', 'r') as f:
            loaded = json.load(f)
            print(loaded)
```

```
{'name': 'Dennis', 'course': 'Python for Data Science'}
```

Read the CSV File

This code imports the pandas library, reads a CSV file named **GSSsubset.csv** into a DataFrame, and stores it in `df`. The `df.head()` function then displays the first five rows of the dataset to give a quick preview of the data.

```
In [10]: import pandas as pd
        df = pd.read_csv("GSSsubset.csv")
        df.head()
```

```
Out[10]:
```

	id	sex	degree	income	marital	age	height	weight	hrrwrk
0	1	MALE	BACHELOR	60967.50	DIVORCED	53.0	72.0	190.0	60.0
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26.0	60.0	97.0	40.0
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56.0	68.0	160.0	20.0
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40.0	65.0	156.0	37.0
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56.0	NaN	210.0	6.0

Jupyter Notebooks

Jupyter Notebooks provide an interactive environment for writing and running Python code, making it ideal for data science. Best practices include:

- Use Markdown cells to document code
- Break long notebooks into sections with headers
- Avoid running long or destructive operations without checks
- Keep output clean and reproducible

Practical Lab: Data Cleaning Without Libraries

Objective: Use core Python to clean a mock dataset loaded from a text file, including stripping whitespace, filtering missing entries, and computing basic stats.

```
In [11]: # Sample data (normally loaded from file)
        data = [
            " Name , Age , Score ",
```

```

    " Alice , 24 , 88 ",
    " Bob , , 75 ",
    " Charlie , 22 , 90",
    " Dana , 27 , 85 "
]
data

```

```

Out[11]: [' Name , Age , Score ',
          ' Alice , 24 , 88 ',
          ' Bob , , 75 ',
          ' Charlie , 22 , 90',
          ' Dana , 27 , 85 ']

```

```
header = [col.strip() for col in data[0].split(',')]
```

- Take the first row of `data` (the header row).
- Split the string by commas to separate column names.
- Use `.strip()` to remove extra spaces from each column name.
- Store the clean column names in the list `header`.

```
records = []
```

- Create an empty list `records` to store the cleaned data rows as dictionaries.

```
for row in data[1:]:
```

- Loop over all rows in `data` **except the header** (i.e., from the second row onward).

```
fields = [field.strip() for field in row.split(',')]
```

- Split the current row string by commas to get individual field values.
- Strip leading and trailing spaces from each field.
- Store the cleaned values in the list `fields`.

```
if all(fields):
```

- Check if **all fields have a value** (i.e., no empty or missing data).
- Only proceed if the row is complete.

```
records.append(dict(zip(header, fields)))
```

- Pair each field value with its corresponding column name using `zip(header, fields)`.
- Convert the zipped pairs into a dictionary.
- Append the dictionary to the `records` list.

```
print("Cleaned Records:")
```

```
for rec in records:
    print(rec)
```

- Print the title `"Cleaned Records:"`.
- Loop through each dictionary in `records` and print it to show the cleaned data.

```
In [12]: # Clean and process data
header = [col.strip() for col in data[0].split(',')]
records = []
for row in data[1:]:
    fields = [field.strip() for field in row.split(',')]
    if all(fields):
        records.append(dict(zip(header, fields)))

print("Cleaned Records:")
for rec in records:
    print(rec)
```

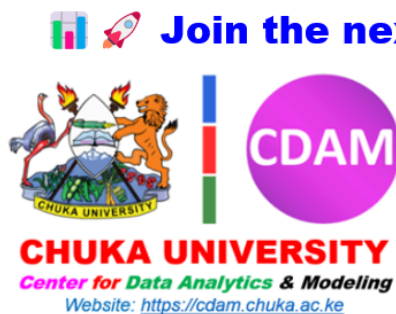
Cleaned Records:

```
{'Name': 'Alice', 'Age': '24', 'Score': '88'}
{'Name': 'Charlie', 'Age': '22', 'Score': '90'}
{'Name': 'Dana', 'Age': '27', 'Score': '85'}
```

Finally, the program extracts the Score values from the cleaned records, converts them to integers, computes the average score, and prints the result.

```
In [13]: # Compute average score
scores = [int(rec['Score']) for rec in records]
avg_score = sum(scores) / len(scores)
print("Average Score:", avg_score)
```

Average Score: 87.66666666666667



  **Join the next generation of data leaders!**

Contact

 **Location:** Chuka University, CDAM-Lab, Kenya

 **Phone:** +254735885755

 **Email:** dataanalytics@chuka.ac.ke

 **Website:** <https://cdam.chuka.ac.ke>

 **LinkedIn:** <https://www.linkedin.com/in/dennis-k-muriithi-6768414a/>